

Tổng Hợp Kiến Thức

Server Cơ Bản (`http.createServer`)

```
const http = require('http');

const server = http.createServer((req, res) => {
  // Xử lý request tại đây
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server chạy tại <http://localhost>:${PORT}`);
});
```

- `http.createServer((req, res) => { ... })` : Tạo server, callback xử lý mỗi request.
 - `req` (request): Thông tin request từ client (URL, method, headers, body).
 - `res` (response): Gửi phản hồi về client.
- `res.writeHead(statusCode, headers)` : Thiết lập status code (ví dụ `200` , `404`) và headers (ví dụ `Content-Type`).
- `res.end(body)` : Gửi body response và kết thúc response.
- `server.listen(PORT, callback)` : Bắt đầu server lắng nghe ở cổng `PORT` .

Xử lý Path:

```
if (req.url === '/') {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Trang chủ');
} else if (req.url === '/about') {
  // ...
} else {
  res.writeHead(404, { 'Content-Type': 'text/plain' });
  res.end('Không tìm thấy');
}
```

Status Code phổ biến:

- `200 OK` : Thành công.
- `201 Created` : Tạo mới thành công (POST).
- `204 No Content` : Xóa thành công (DELETE).
- `400 Bad Request` : Request không hợp lệ (lỗi client).
- `404 Not Found` : Không tìm thấy tài nguyên.
- `500 Internal Server Error` : Lỗi server.

Content-Type phổ biến:

- `text/plain` : Văn bản thuần túy.
- `text/html` : Trang HTML.
- `application/json` : Dữ liệu JSON.

Postman & RESTful API

- **Postman**: Công cụ test API. Gửi request, xem response trực quan.
- **RESTful API**: Bộ quy tắc thiết kế API chuẩn mực.
 - **Tài nguyên (Resources)**: Mọi thứ là "tài nguyên" (products, users...). Gọi bằng **URL** (danh từ, số nhiều). Ví dụ: `/products` , `/users` .

- **Phương thức HTTP (HTTP Methods):** "Động từ" hành động trên tài nguyên.
 - **GET:** Lấy dữ liệu (đọc - Read).
 - **POST:** Tạo mới dữ liệu (tạo - Create).
 - **PUT/PATCH:** Cập nhật dữ liệu (cập nhật - Update).
 - **DELETE:** Xóa dữ liệu (xóa - Delete).
- **JSON:** "Ngôn ngữ" dữ liệu phổ biến cho API.

API GET (`/products`) - Lấy Danh Sách

- `data.json` : File chứa dữ liệu JSON (ví dụ danh sách sản phẩm).
- **Đọc file JSON (bất đồng bộ):**

```
const fs = require('fs').promises;

async function getProducts(req, res) {
  try {
    const data = await fs.readFile('data.json', 'utf8');
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(data);
  } catch (error) {
    res.writeHead(500, { 'Content-Type': 'text/plain' });
    res.end('Lỗi server');
  }
}

// ... trong createServer ...
if (req.url === '/products' && req.method === 'GET') {
  getProducts(req, res);
}
```

API POST (`/products`) - Thêm Mới

- **Đọc Request Body (stream):**

```
let body = '';
req.on('data', chunk => { body += chunk; });
req.on('end', async () => {
  // body chứa dữ liệu request
  try {
    const newProduct = JSON.parse(body);
    // ... xử lý và ghi file ...
  } catch (error) {
    res.writeHead(400, { 'Content-Type': 'text/plain' });
    res.end('Dữ liệu không hợp lệ');
  }
});
```

- **Ghi file JSON (bất đồng bộ):** `fs.promises.writeFile('data.json', JSON.stringify(products, null, 2))`

API PUT/PATCH (`/products/{id}`) - Cập Nhật

- Lấy ID từ URL: `const productId = parseInt(req.url.split('/')[2]);`
- Tìm sản phẩm trong mảng (dựa vào ID): `products.findIndex(p => p.id === productId);`
- Cập nhật object: `products[productIndex] = { ...products[productIndex], ...updatedProduct, id: productId };`

API DELETE (`/products/{id}`) - Xóa

- Lấy ID từ URL (tương tự PUT/PATCH).
- Tìm sản phẩm trong mảng (dựa vào ID).
- Xóa khỏi mảng: `products.splice(productIndex, 1);`
- Response status code `204 No Content` : Thành công, không có body response.

Đặt Tên API & Bảo Trì

- **Nguyên tắc đặt tên URL:**
 - Danh từ số nhiều cho resources (ví dụ: `/products`).
 - HTTP methods thể hiện hành động (GET, POST...).
 - URL parameters cho ID (ví dụ: `/products/{id}`).
 - Tránh động từ trong URL (ví dụ: không dùng `/getProducts`).
 - Nhất quán.
- **Refactoring:** Tách logic endpoint vào function riêng (ví dụ `getProducts()` , `createProduct()` ,...).
- **Bảo trì & Mở rộng:** Code rõ ràng, cấu trúc tốt giúp dễ bảo trì và nâng cấp. Framework (ví dụ Express.js) cho dự án phức tạp hơn.

CORS (Cross-Origin Resource Sharing)

- **Same-origin policy:** Trình duyệt chặn request cross-origin (khác domain, port, protocol) mặc định vì bảo mật.
- **CORS:** Cơ chế server cho phép origin khác truy cập API.
- **Cấu hình CORS (headers):**

```
res.setHeader('Access-Control-Allow-Origin', '*'); // Cho phép mọi origin (DEMO ONLY)
res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization');

if (req.method === 'OPTIONS') { // Xử lý Preflight request
  res.writeHead(204);
  res.end();
  return;
}
```

- `Access-Control-Allow-Origin` : Origin được phép (ví dụ `'<http://localhost:5500>'` hoặc `'*'`). **KHÔNG DÙNG `'*'` TRONG PRODUCTION.**
- `Access-Control-Allow-Methods` : HTTP methods được phép.
- `Access-Control-Allow-Headers` : Headers được phép.
- **Preflight request (OPTIONS):** Trình duyệt gửi OPTIONS trước request "phức tạp" (POST, PUT, DELETE...) để kiểm tra CORS. Server cần xử lý OPTIONS.